第7章 函数 —— 课后作业答案

基础练习

1. 选择题

题号	答案	解析
(1)	**C**	A 缺少参数类型声明；B 缺少返回值类型；D 第二个参数 `b` 缺少类型。C 是标准写法
(2)	**B**	递归函数必须有终止条件（基线条件），否则会无限递归导致栈溢出

2. 程序填空

```
int isLeapYear(int year)
{
    return (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
    // 填空: ||
}
```

解析：闰年条件 =（能被4整除且不能被100整除）**或**（能被400整除）

编程练习

3. 求最大公约数函数

```
#include <stdio.h>

// 辗转相除法求最大公约数
int gcd(int a, int b)
{
    int temp;
    while (b != 0)
    {
        temp = a % b;
        a = b;
        b = temp;
    }
    return a;
}

int main()
{
    int x, y;
    printf("请输入两个正整数: ");
    scanf("%d %d", &x, &y);
```

```
    printf("最大公约数 = %d\n", gcd(x, y));
    return 0;
}
```

---

4. 哥德巴赫猜想验证

```
#include <stdio.h>

// 判断素数
int isPrime(int n)
{
    if (n <= 1) return 0;
    for (int i = 2; i * i <= n; i++)
    {
        if (n % i == 0)
            return 0;
    }
    return 1;
}

int main()
{
    int even;
    printf("请输入一个大于2的偶数: ");
    scanf("%d", &even);

    if (even <= 2 || even % 2 != 0)
    {
        printf("输入必须是大於2的偶数!\n");
        return 1;
    }

    for (int i = 2; i <= even / 2; i++)
    {
        if (isPrime(i) && isPrime(even - i))
        {
            printf("%d = %d + %d\n", even, i, even - i);
            break;  // 找到一组即可
        }
    }
    return 0;
}
```

运行示例：输入 `20` → 输出 `20 = 3 + 17`

---

5. 递归求斐波那契数列

```
#include <stdio.h>
```

```c
int fib(int n)
{
    if (n == 1 || n == 2)
        return 1;              // 终止条件
    else
        return fib(n - 1) + fib(n - 2);  // 递归
}

int main()
{
    int n;
    printf("求第几项斐波那契数? ");
    scanf("%d", &n);

    printf("fib(%d) = %d\n", n, fib(n));

    // 思考：为什么递归效率低？
    // 因为 fib(5) = fib(4) + fib(3)
    //             = [fib(3)+fib(2)] + [fib(2)+fib(1)]
    //             = [[fib(2)+fib(1)]+fib(2)] + [fib(2)+fib(1)]
    // fib(2) 被重复计算了 3 次！n 越大，重复越严重，复杂度 O(2^n)
    // 迭代法只需 O(n)，更高效。
    return 0;
}
```

6. 字符统计

```c
#include <stdio.h>
#include <string.h>

void countChars(char str[])
{
    int upper = 0, lower = 0, digit = 0, space = 0;
    int i, len = strlen(str);

    for (i = 0; i < len; i++)
    {
        if (str[i] >= 'A' && str[i] <= 'Z')
            upper++;
        else if (str[i] >= 'a' && str[i] <= 'z')
            lower++;
        else if (str[i] >= '0' && str[i] <= '9')
            digit++;
        else if (str[i] == ' ')
            space++;
    }

    printf("大写字母: %d\n", upper);
    printf("小写字母: %d\n", lower);
    printf("数字: %d\n", digit);
    printf("空格: %d\n", space);
```

```
}

int main()
{
    char str[200];
    printf("请输入一段文字: ");
    gets(str);
    countChars(str);
    return 0;
}
```

---

7. 矩阵运算（多文件）

matrix.h（头文件）：

```
#ifndef MATRIX_H
#define MATRIX_H

#define N 3  // 假设3×3矩阵，可修改

// 矩阵加法: C = A + B
void matAdd(int A[N][N], int B[N][N], int C[N][N]);

// 矩阵乘法: C = A × B
void matMul(int A[N][N], int B[N][N], int C[N][N]);

// 打印矩阵
void matPrint(int mat[N][N]);

#endif
```

matrix.c（实现）：

```
#include <stdio.h>
#include "matrix.h"

void matAdd(int A[N][N], int B[N][N], int C[N][N])
{
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            C[i][j] = A[i][j] + B[i][j];
}

void matMul(int A[N][N], int B[N][N], int C[N][N])
{
    for (int i = 0; i < N; i++)
    {
        for (int j = 0; j < N; j++)
        {
            C[i][j] = 0;
            for (int k = 0; k < N; k++)
```

```
                C[i][j] += A[i][k] * B[k][j];
        }
    }
}

void matPrint(int mat[N][N])
{
    for (int i = 0; i < N; i++)
    {
        for (int j = 0; j < N; j++)
            printf("%4d ", mat[i][j]);
        printf("\n");
    }
}
```

main.c（测试）：

```
#include <stdio.h>
#include "matrix.h"

int main()
{
    int A[N][N] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
    int B[N][N] = {{9, 8, 7}, {6, 5, 4}, {3, 2, 1}};
    int C[N][N];

    printf("A = \n"); matPrint(A);
    printf("\nB = \n"); matPrint(B);

    matAdd(A, B, C);
    printf("\nA + B = \n"); matPrint(C);

    matMul(A, B, C);
    printf("\nA × B = \n"); matPrint(C);

    return 0;
}
```

编译命令：`gcc main.c matrix.c -o program`